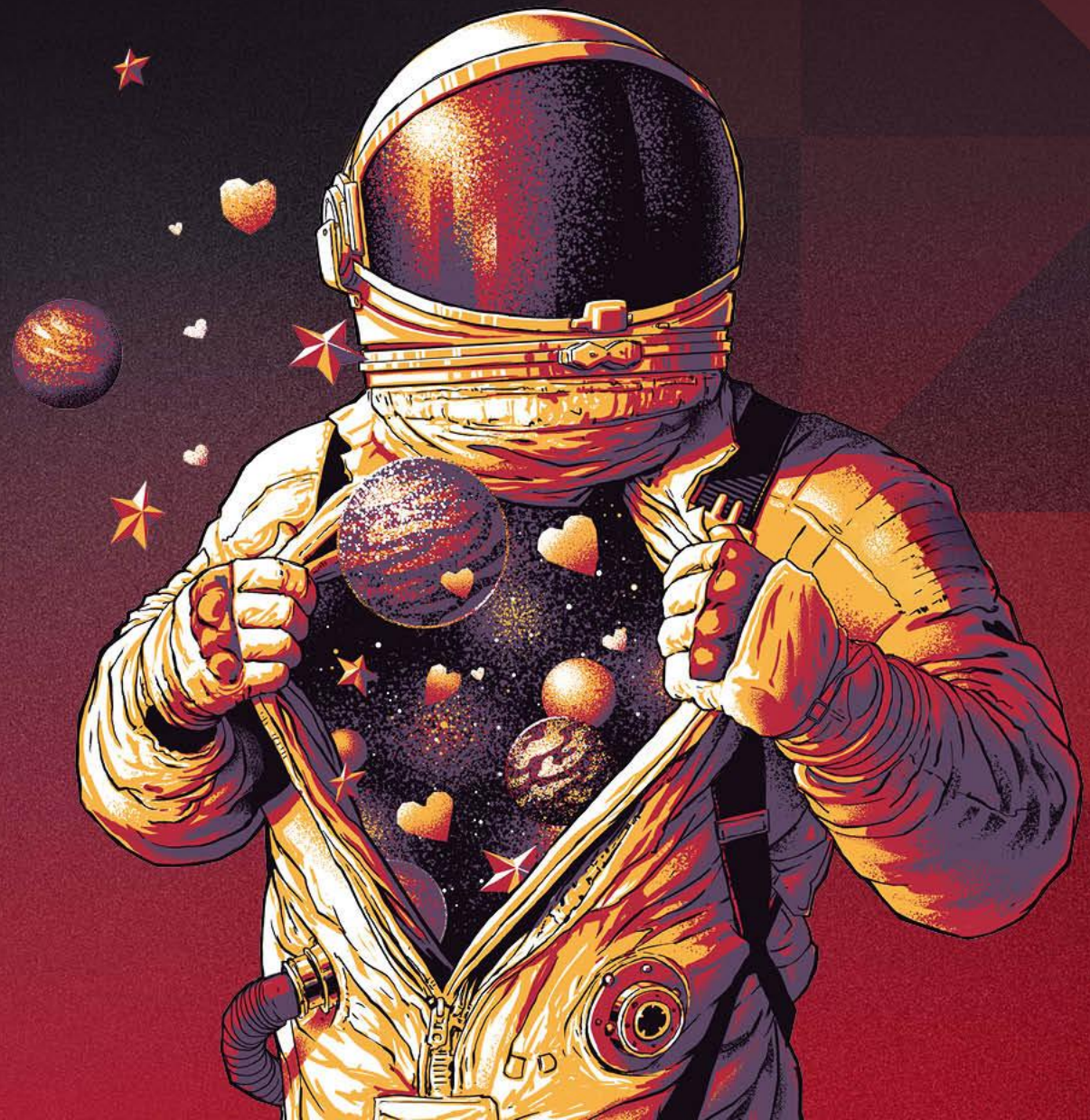
Designing Production-Grade Agentic Systems

From Prototype to Scalable Infrastructure

Eva Donaldson · Enterprise Support Manager, AWS

Siddhi Patil · Senior Business Intelligence Engineer, AWS



re:Invent 2026

OCTOBER 19-20

Raleigh, NC USA

Meet your speakers



Eva Donaldson

Enterprise Support Manager, AWS

Specializing in Healthcare & Life Sciences customers. With 20+ years of experience as a data architect, engineer, and team manager, she focuses on designing automated data platforms and solutions that solve real business problems.

linkedin.com/in/eva-donaldson-5b72171 · github.com/evaleah



Siddhi Patil

Senior Business Intelligence Engineer, AWS

Builds data products and AI agents that turn messy, multi-source data into reusable, AI-ready foundations. Co-leads the Amazon Women in Engineering RTP chapter and speaks regularly at tech conferences.

linkedin.com/in/siddhi-patil

Everyone has seen the agent demo that worked… once.

The model isn't the hard part. A reusable, AI-ready data foundation is.

Today: Prototype → Data foundation → Scale

Agenda

01

The prototype trap

Why demos break

02

Modular workflows + semantic layer

Build once, use everywhere

03

Evaluation & guardrails

Catch hallucinations, stop leaks

04

Observability & cost

See it, size it, scale it

05

Recorded demo

Conference Schedule Concierge

06

Blueprint + takeaways

Your production checklist

One giant prompt doesn't survive production

THE PROMPT-STUFFED PROTOTYPE

```
You are a helpful conference assistant.
The AI track is in Workshop Room C.
Speakers are in speakers.csv; join on session_id.
The panel is probably in Hall A?
If unsure, make a best guess.
… 200 more lines …
```

1. **Drifts on every edit**

   One change to the schedule means rewriting the prompt.

2. **Not versioned, not tested**

   No review, no history, no way to know what broke.

3. **Doesn't travel**

   Every tool or host needs its own copy.

When knowledge lives in the prompt, every change is a redeploy and every host needs its own copy.

A layered, extensible platform

Each layer depends only on the one below it. Build the foundation once; add a source, a tool, or a whole capability by editing a layer — not rewriting the agent.

CLI — **main.py · thin entry**
Asks the agent a question. No logic of its own.

L4 — **agent.py — model + prompt**
Strands Agent wiring and a system prompt that carries the rules, not the structure.

L3 — **orchestration.py — tools + join engine**
The two tools, the deterministic join engine, and inline guardrails.

L2 — **semantic.py + semantic/*.md — governed layer**
Versioned markdown: source catalog, term → field map, join + correctness rules. Read at runtime.

L1 — **data_platform.py — reusable foundation**
Domain-agnostic: read a source's rows. No business words — reusable under any app.

each layer reads only the one below

Swap or extend any layer on its own — that is what makes the platform extensible and production-ready.

Trust is something you test

Evaluate

- Contract tests: right keys, cited sources, partial results flagged
- Scored answer quality, run in CI on every change
- “Never invent” rules + partial-result flags catch hallucinations early

Secure

- Prompt injection: treat tool output as data, not instructions
- Least-privilege tools: only the access each step needs
- Data leakage: authorization stays at the source; the layer holds no data

In the demo, “room TBD” is a guardrail you can write a test for.

Observability & cost at scale

Same agent, same layer: laptop → container or serverless → shared gateway.

What changes at scale is what you watch and what you pay for.

Traces

Every tool call, with latency per step

Drift

Answer quality vs. layer version, over time

Right-size

Smaller models for simple steps

Context

The layer and repeat lookups; keep context small

03 · LIVE DEMO

Conference Schedule Concierge Demo

Fictional DevConf 2026 · sessions · tracks · rooms · speakers

Built on the open-source Strands Agents SDK. The pattern works with any framework.

OCTOBER 19-20
Raleigh, NC USA

The production blueprint

Modular tools
Small, testable, swappable

Served semantic layer
Knowledge as versioned content

Eval in CI
Contract tests + scored quality

Guardrails
Injection, least privilege, no leaks

Observability
Traces, latency, drift

Cost controls
Right-size, cache, small context

WHAT WE'D TELL OUR PAST SELVES

Start with the eval, not the prompt.

If it's knowledge, it's content. Version it.

Design for “I don't know.”

RESOURCES and THANK YOU

Code: github.com/aws-samples/sample-ai-agent-poc-to-prod

Provide feedback using ATO26 app
OR
Scan the QR code

Connect with us on LinkedIn

Eva Donaldson
@linkedin.com/in/eva-donaldson-5b72171

Siddhi Patil
@linkedin.com/in/siddhi-patil